

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

- 1 Sériová komunikace
- 2 ASCII driver - Control Web

Téma

- 1 Sériová komunikace
- 2 ASCII driver - Control Web

Sériová komunikace

- způsob přenosu dat, při kterém se informace posílají postupně po jednotlivých bitech za sebou,
- jako komunikační kanál v jednom směru často dostačuje jen jeden vodič.

Základní princip:

- Odeslání binární hodnoty $(10110010)_2$,
- na komunikační kanál vysílač postupně se stanovenou periodou zadává hodnoty:

$1 \longrightarrow 0 \longrightarrow 1 \longrightarrow 1 \longrightarrow 0 \longrightarrow 0 \longrightarrow 1 \longrightarrow 0$

- přijímač i vysílač musí nějakým způsobem stanovit časový interval pro vyslání a čtení hodnoty v komunikačním kanálu.

Sériová komunikace - výhody

- méně vodičů,
- jednodušší zapojení,
- levnější hardware,
- možnost komunikace na větší vzdálenosti,
- menší citlivost na rušení.

Poslední dva body platí především pro linky využívající proudovou smyčku nebo diferenciální pár.

Rozlišujeme:

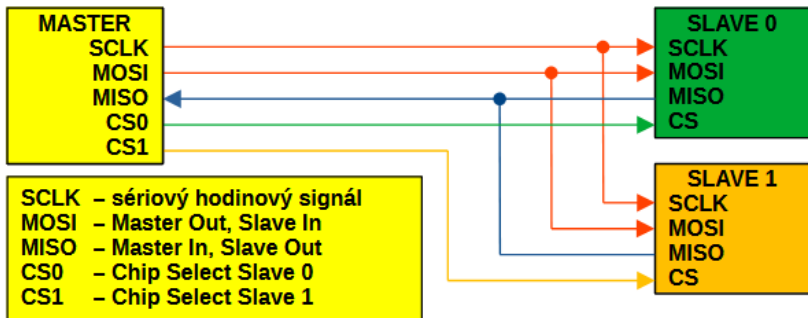
- **synchronní** komunikaci - zařízení sdílí společný hodinový signál (SPI, I²C),
- **asynchronní** komunikaci - bez sdíleného hodinového signálu (RS232, RS485).

SPI

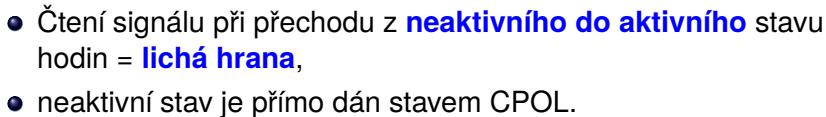
Serial Peripheral Interface

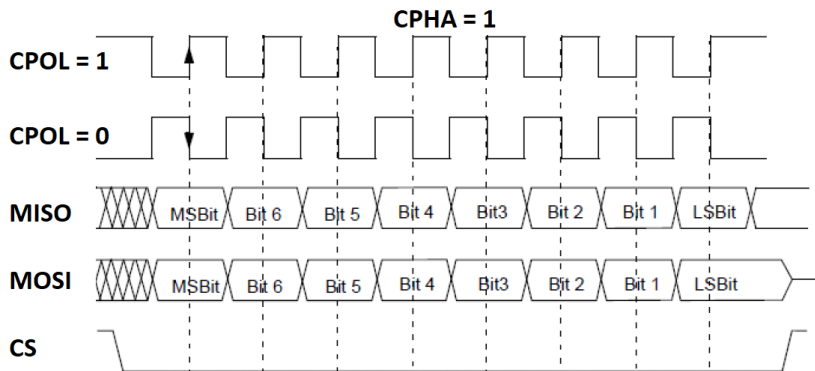
- komunikace s paměťmi a senzory,
- jeden obvod je řídicí - Master a vystavuje řídicí signály komunikace,
- ostatní obvody, jsou typu Slave,
- Full-Duplex komunikace - oběma směry se komunikuje v jeden moment,
- je určena pro kratší komunikační vzdálenosti,
- čtení i vyslání bitu je řízeno hodinovým signálem, který je součástí komunikace,
- rychlost komunikace definuje Master pomocí hodinového signálu a je omezena jen fyzikálními limity komunikační linky.

SPI - zapojení



- Lze komunikovat s více Slave jednotkami na jedné sběrnici, pokud existuje příslušný CS signál.

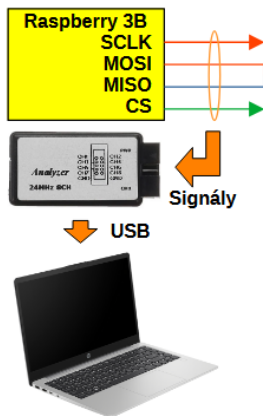


$$\text{SPI} - \text{CPHA} = 1$$


- Čtení signálu při přechodu z **aktivního do neaktivního** stavu hodin = **sudá hrana**,
- neaktivní stav je přímo dán stavem CPOL.

SPI - ukázka, loopback

Zapojení:

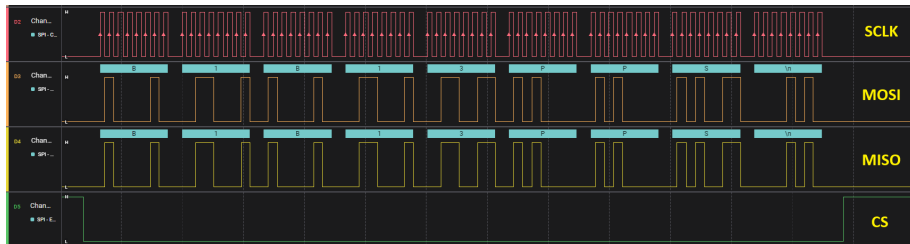


RPI program:

```

11 import spidev
12
13 class SPIExample:
14     """
15     Objekt ukázkového příkladu SPI komunikace
16     """
17     def __init__(self):
18         """
19         Nastavení SPI komunikace
20         """
21         self._dev = spidev.SpiDev()
22         self._dev.open(bus=0, device=1)
23         self._dev.max_speed_hz = 100000
24         self._dev.mode = 0b00
25
26     def __del__(self):
27         """
28         Uzavření komunikace
29         """
30         if self._dev is not None:
31             self._dev.close()
32
33     def transaction_call(self):
34         int_list = list(b'B1B13PPS\n')
35         response = self._dev.xfer2(int_list)
36         print("Přijata zpráva: {}".format(bytes(response).decode('ascii')))
37
38
39 if __name__ == '__main__':
40     obj = SPIExample()
41     obj.transaction_call()
42 
```

SPI - logický analyzátor



- Znaky jsou kódovány pomocí ASCII,
- CPOL=0, klidová hodnota hodin je 0
- CPHA=0, lichá hrana, přechod z 0 do 1

ASCII tabulka

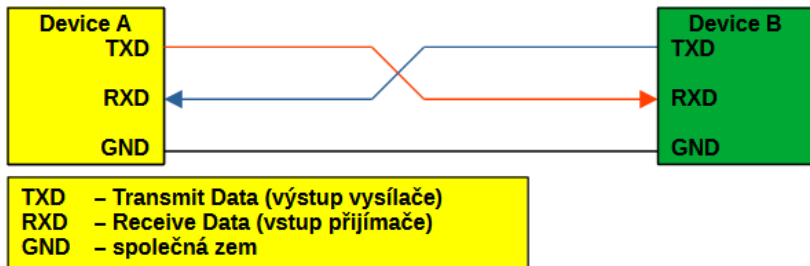
Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
00	NUL	10	DLE	20	SP	30	0	40	@	50	P	60	`	70	p
01	SOH	11	DC1	21	!	31	1	41	A	51	Q	61	a	71	q
02	STX	12	DC2	22	"	32	2	42	B	52	R	62	b	72	r
03	ETX	13	DC3	23	#	33	3	43	C	53	S	63	c	73	s
04	EOT	14	DC4	24	\$	34	4	44	D	54	T	64	d	74	t
05	ENQ	15	NAK	25	%	35	5	45	E	55	U	65	e	75	u
06	ACK	16	SYN	26	&	36	6	46	F	56	V	66	f	76	v
07	BEL	17	ETB	27	'	37	7	47	G	57	W	67	g	77	w
08	BS	18	CAN	28	(	38	8	48	H	58	X	68	h	78	x
09	HT	19	EM	29	)	39	9	49	I	59	Y	69	i	79	y
0A	LF	1A	SUB	2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
0B	VT	1B	ESC	2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
0C	FF	1C	FS	2C	,	3C	<	4C	L	5C	\	6C	l	7C	
0D	CR	1D	GS	2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
0E	SO	1E	RS	2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
0F	SI	1F	US	2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

UART

Universal Asynchronous Receiver-Transmitter

- Základ pro:
 - standard RS232, RS422: komunikace mezi dvěma rovnocennými zařízeními,
 - standard RS485: komunikace mezi mnoha zařízeními (**Half-Duplex**)
- počátky v 60. letech (1969 RS232C),
- formát se stále používá v průmyslu - Modbus RTU, M-Bus, ...
- základem je obousměrná komunikace po 2/3 vodičích,
- RS232 používá navíc řídicí vodiče RTS, CTS, DTR, RI, DCD,
- standardy se liší v napěťových úrovních:
 - **RS232**: log. 0: -15V až -3V, log. 1: 3V až 15V
 - **TTL**: log. 0: 0V, log. 1: 5V

UART - zapojení

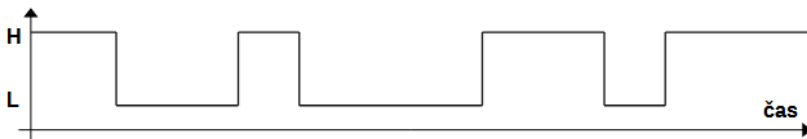
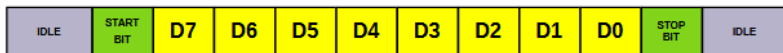


- Bez vodiče sdíleného hodinového signálu:
 - musí dojít k synchronizaci přijímače s vysílanými daty,
 - se musí shodovat přenosová rychlost,
 - se musí shodovat rámec přenášených dat (např. počet bitů).

UART - datový rámeček

Rámeček pro 8 datových bitů, 1 stop bit a bez parity:

Přenášený znak: „F“ (ASCII 0x46 = 01000110₂)



START BIT	Označuje začátek rámečku - hrana
D7:0	Data, pořadí MSB → LSB
STOP BIT	Linka je v klidové úrovni po dobu 1 bitu

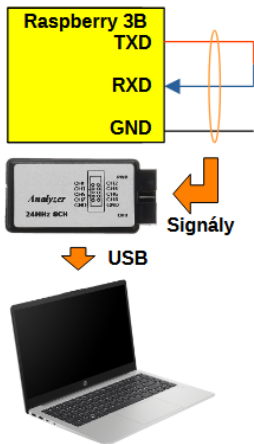
UART - vlastnosti

- **Baud (Bd)**: jednotka modulační rychlosti, počet změn stavu přenosového média za sekundu,
- standardizované rychlosti: 150, 300, 600, 1200, 2400, 4800, 9600, 19200, 38400, 57600 a 115200 Bd
- vždy 1 start bit,
- pořadí dat:
 - **MSB→LSB**
 - **LSB→MSB**
- možnost paritního bitu:
 - sudá parita doplňuje počet 1 na sudý počet,
 - lichá parita doplňuje počet jedniček na lichý počet,
- 1, 1.5 nebo 2 stop bity.

UART - ukázka, loopback

Zapojení:

RPI program:

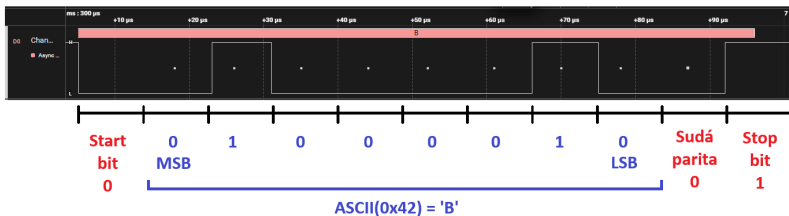
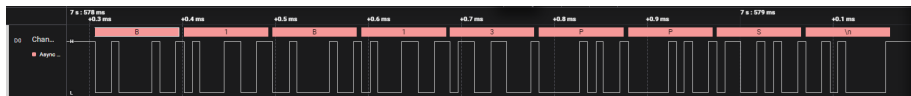


```

11 import serial
12
13 class SerialExample:
14     """
15     Objekt ukázkového příkladu sériové komunikace
16     """
17     def __init__(self):
18         """
19         Nastavení sériové komunikace
20         """
21         self.__dev = serial.Serial(port='/dev/serial0',
22                                   baudrate=115200,
23                                   parity=serial.PARITY_EVEN,
24                                   stopbits=serial.STOPBITS_ONE,
25                                   timeout=2)
26         self.__bytes_cnt = 0
27
28     def __del__(self):
29         """
30         Uzavření komunikace
31         """
32         if self.__dev is not None and self.__dev.is_open:
33             self.__dev.close()
34
35     def transaction_call(self):
36         self.__bytes_cnt = self.__dev.write(b'B1B13PPS\n')
37         self.__dev.flush()
38         response = self.__dev.read(size=self.__bytes_cnt)
39         print("Přijata zpráva: {}".format(response))
40
41
42 if __name__ == '__main__':
43     obj = SerialExample()
44     obj.transaction_call()

```

UART - logický analyzátor



- Jednodušší komunikace než SPI,
- formát přenosu musí být nastaven dopředu na obou stranách.

Téma

1 Sériová komunikace

2 ASCII driver - Control Web

UART v PC

- Moderní PC nemají port RS232 ani UART,
- v praxi se používají převodníky pro USB.

USB-UART-TTL



USB-RS232



Následující přednáška

- Komunikační rozhraní a jejich použití v průmyslu

Děkuji za pozornost